

Week 5 - Lambda Calculus

Notation: $\lambda xy. x$ is shorthand for $\lambda x. \lambda y. x$.

Exercise 1: I-Combinator

What is it, why use it, and $x \Rightarrow x$ in lambda calculus:

$x \Rightarrow x \equiv \lambda x. x$ is the identity for composition.

In JS

```
const compose = f => g => x => f(g(x));
```

And we know it's the identity because

```
compose(f, I) = x => f(I(x)) = x => f(x) = f
compose(I, f) = x => I(f(x)) = x => f(x) = f
```

The identity when applied to anything just disappears, because when we substitute an argument for x in the body x , it's just the argument.

The reason it's a combinator is because there are no free variables, x is bound by its own λ .

Exercise 2: Alpha equivalence

Alpha equivalence means consistently renaming a *bound* variable throughout its own scope. Free variables can't be touched, and a rename that turns a free variable into a bound one (capture) is not allowed.

1. Alpha equivalent to $\lambda x. x$:

- a. $\lambda x. y$
- b. $\lambda a. a$
- c. $\lambda z. x$

Answer: b

2. Alpha equivalent to $\lambda xy. yx$:

- a. $\lambda az. az$
- b. $\lambda a. (\lambda b. ba)$
- c. $\lambda az. ba$

Answer: b, $\lambda xy. yx = \lambda x. \lambda y. yx = \lambda x. (\lambda y. yx) = \lambda a. (\lambda b. ba)$

3. Alpha equivalent to $\lambda xy. xz$:

- a. $\lambda xz. xz$
- b. $\lambda mn. mz$
- c. $\lambda z. (\lambda x. xz)$

Answer: b, $\lambda xy. xz \lambda mn. mz$, where $x \rightarrow m$, $y \rightarrow n$, $z \rightarrow z$

Exercise 3: Beta normal form or divergence?

1. $(\lambda x. x)y$

- binder: x, body: x, argument: y
- y is a plain variable so nothing to do
- taking the body x
- swap with argument y
- so you get $x \rightarrow y$

This is normal form.

2. $\lambda x. xx$ beta normal form, no redex.

3. $(\lambda z. zz)(\lambda y. yy)$

- binder: z, body: zz, argument: $(\lambda y. yy)$
- swap every z in the body for $(\lambda y. yy)$
- $(\lambda y. yy)(\lambda y. yy)$
- that's a redex again: binder y, body yy, argument $(\lambda y. yy)$
- $(\lambda y. yy)(\lambda y. yy)$

We're back to exactly where we started, so this never settles. This diverges (it's Ω , the classic looping term).

4. $(\lambda x. xx)y$

- binder: x, body: xx, argument: y
- swap every x in the body for y
- yy

y is a free variable, not a lambda, so yy has no redex to reduce.

This is normal form.

Exercise 4: Beta reduction

1. $(\lambda y. zy)a$

- binder: y, body: zy, argument: a
- swap y for a
- za

2. $(\lambda x. x)(\lambda x. x)$

- alpha rename the argument first so the two x's don't get confused: $(\lambda x.x)(\lambda a.a)$
- binder: x, body: x, argument: $(\lambda a.a)$
- swap x for $(\lambda a.a)$
- $\lambda a.a$

Applying I to I just gives you I back, which is what you'd expect from identity.

3. $(\lambda x.xy)(\lambda x.xx)$

- binder: x, body: xy, argument: $(\lambda x.xx)$
- swap x for $(\lambda x.xx)$
- $(\lambda x.xx)y$
- redex again: binder x, body xx, argument y
- yy

4. $(\lambda z.z)(\lambda a.aa)(\lambda z.zb)$

Application is left-associative, so this is $((\lambda z.z)(\lambda a.aa))(\lambda z.zb)$.

- $(\lambda z.z)(\lambda a.aa) \rightarrow \lambda a.aa$
- now $(\lambda a.aa)(\lambda z.zb)$: swap a for $(\lambda z.zb) \rightarrow (\lambda z.zb)(\lambda z.zb)$
- alpha rename the argument: $(\lambda z.zb)(\lambda w.wb)$
- swap z for $(\lambda w.wb)$ in zb $\rightarrow (\lambda w.wb)b$
- swap w for b in wb $\rightarrow bb$

Exercise 5: Eta conversion

Eta says $\lambda x.f x \equiv f$, as long as x isn't free in f. The lambda is just passing its argument straight through, so it's doing nothing.

1. $\lambda x.zx$

The body is z applied to x, and x isn't free in z, so the wrapper is pointless.

$\rightarrow z$

2. $\lambda x.xz$

Can't eta reduce. x is in the function position, not the argument position, so it doesn't match the $\lambda x.f x$ shape.

Already as simple as it gets.

3. $(\lambda x.bx)(\lambda y.ay)$

Eta reduce both sides:

- $\lambda x.bx \rightarrow b$
- $\lambda y.ay \rightarrow a$

$\rightarrow ba$

Exercise 6: Which are combinators?

A combinator is a lambda expression with no free variables, every variable in the body is bound by some λ .

1. $\lambda x. xxx$: yes. x is bound by its own λ .
2. $\lambda xy. zx$: no. z is free, nothing binds it.
3. $\lambda xyz. xy(zx)$: yes. x, y, z are all bound by the three λ s.
4. $\lambda xyz. xy(zxy)$: yes. Same three binders, still nothing free.

Exercise 7: Y-Combinator application

$Y = \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))$, reduce $Y\ g$:

- binder: f , body: $(\lambda x. f(xx))(\lambda x. f(xx))$, argument: g
- swap every f for g
- $(\lambda x. g(xx))(\lambda x. g(xx))$
- that's a redex: binder x , body $g(xx)$, argument $(\lambda x. g(xx))$
- swap every x in $g(xx)$ for $(\lambda x. g(xx))$
- $g((\lambda x. g(xx))(\lambda x. g(xx)))$

But $(\lambda x. g(xx)) (\lambda x. g(xx))$ is exactly what $Y\ g$ reduced to in step 3, so:

```
Y g = g(Y g)
     = g(g(Y g))
     = g(g(g(Y g)))
     = ...
```

It never reaches normal form, it keeps peeling off another g forever. That's the point: g gets handed a copy of itself as an argument, which is how an anonymous function gets to recurse without ever having a name to call.

Exercise 8: Church encoding

```
TRUE  =  $\lambda xy. x$ 
FALSE =  $\lambda xy. y$ 
IF     =  $\lambda btf. b\ t\ f$ 
AND    =  $\lambda xy. IF\ x\ y\ FALSE$ 
OR     =  $\lambda xy. IF\ x\ TRUE\ y$ 
NOT    =  $\lambda x. IF\ x\ FALSE\ TRUE$ 
```

The trick is that **TRUE** and **FALSE** are the selectors themselves. **TRUE** picks its first argument, **FALSE** picks its second, so **IF** doesn't need to inspect anything, it just applies the boolean to the two branches.

1. **NOT FALSE** (should be **TRUE**)

```
NOT FALSE = (λx. IF x FALSE TRUE) FALSE
           = IF FALSE FALSE TRUE
           = (λbtf. b t f) FALSE FALSE TRUE
           = FALSE FALSE TRUE
           = (λxy. y) FALSE TRUE
           = TRUE
```

2. OR TRUE FALSE (should be TRUE)

```
OR TRUE FALSE = (λxy. IF x TRUE y) TRUE FALSE
               = IF TRUE TRUE FALSE
               = (λbtf. b t f) TRUE TRUE FALSE
               = TRUE TRUE FALSE
               = (λxy. x) TRUE FALSE
               = TRUE
```

OR short circuits: once x is TRUE it hands back TRUE without ever looking at y.

3. AND TRUE TRUE (should be TRUE)

```
AND TRUE TRUE = (λxy. IF x y FALSE) TRUE TRUE
               = IF TRUE TRUE FALSE
               = (λbtf. b t f) TRUE TRUE FALSE
               = TRUE TRUE FALSE
               = (λxy. x) TRUE FALSE
               = TRUE
```

Exercise 9: Lambda expressions to JS arrow functions

Each λ is one arrow, one argument, so $\lambda xy. x$ is $\lambda x. \lambda y. x$ is two nested arrows.

1. $\lambda xy. x$

```
const K = x => y => x;
```

This is the K-combinator, same thing as TRUE above.

2. $\lambda fx. f(f\ x)$

```
const twice = f => x => f(f(x));
```

Church numeral 2, applies f to x twice.

3. $\lambda f. \lambda g. \lambda x. f(g\ x)$

```
const compose = f => g => x => f(g(x));
```

Function composition, the same `compose` from Exercise 1.